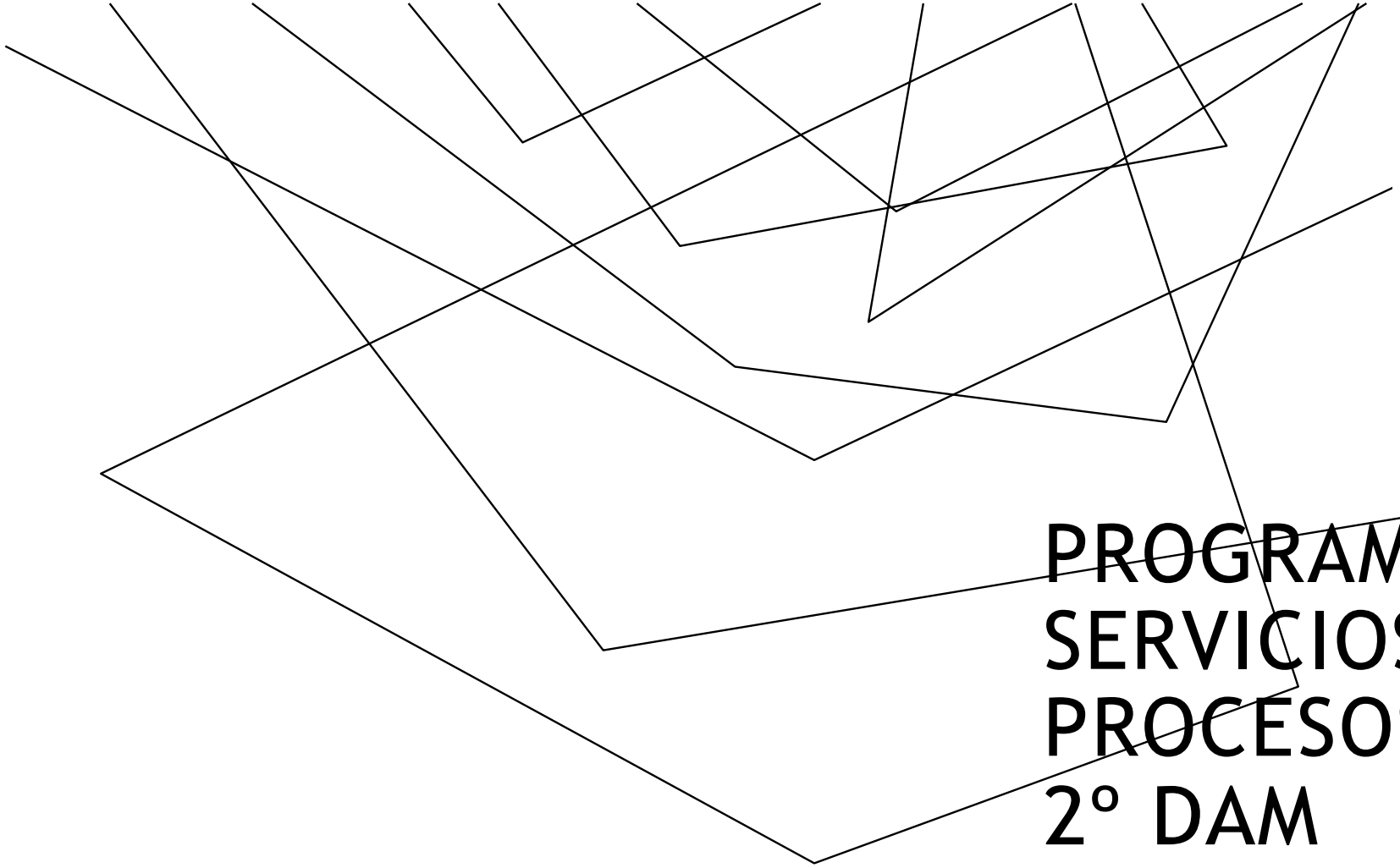


PROGRAMACIÓN DE SERVICIOS Y PROCESOS
2º DAM

Andrés Chillón Sesma

andres.chillon@iesluisbraille.es

UNIDAD 1: **PROCESOS**

CONTENIDOS

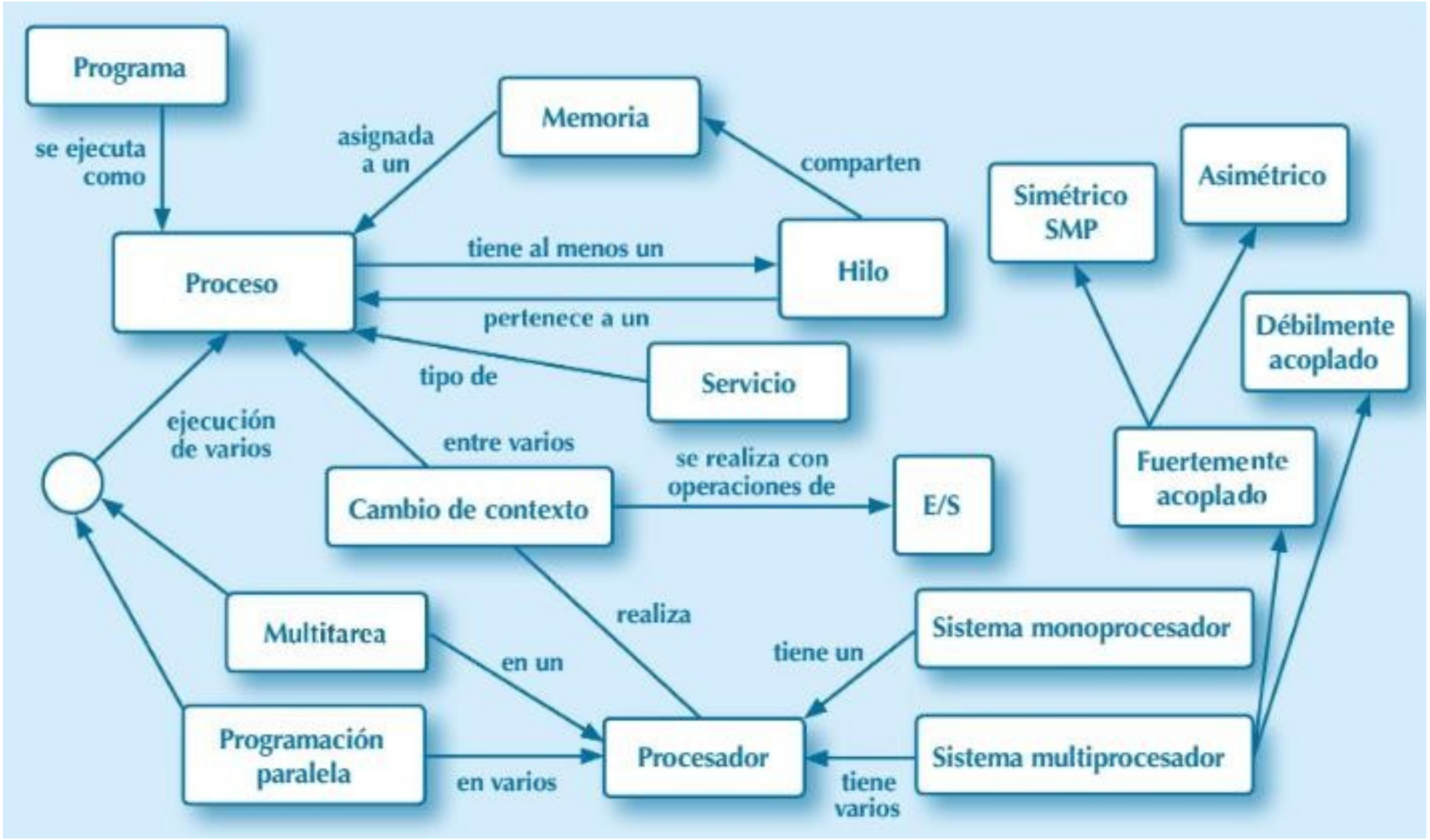
Programas. Ejecutables. Procesos. Servicios.

Procesos:

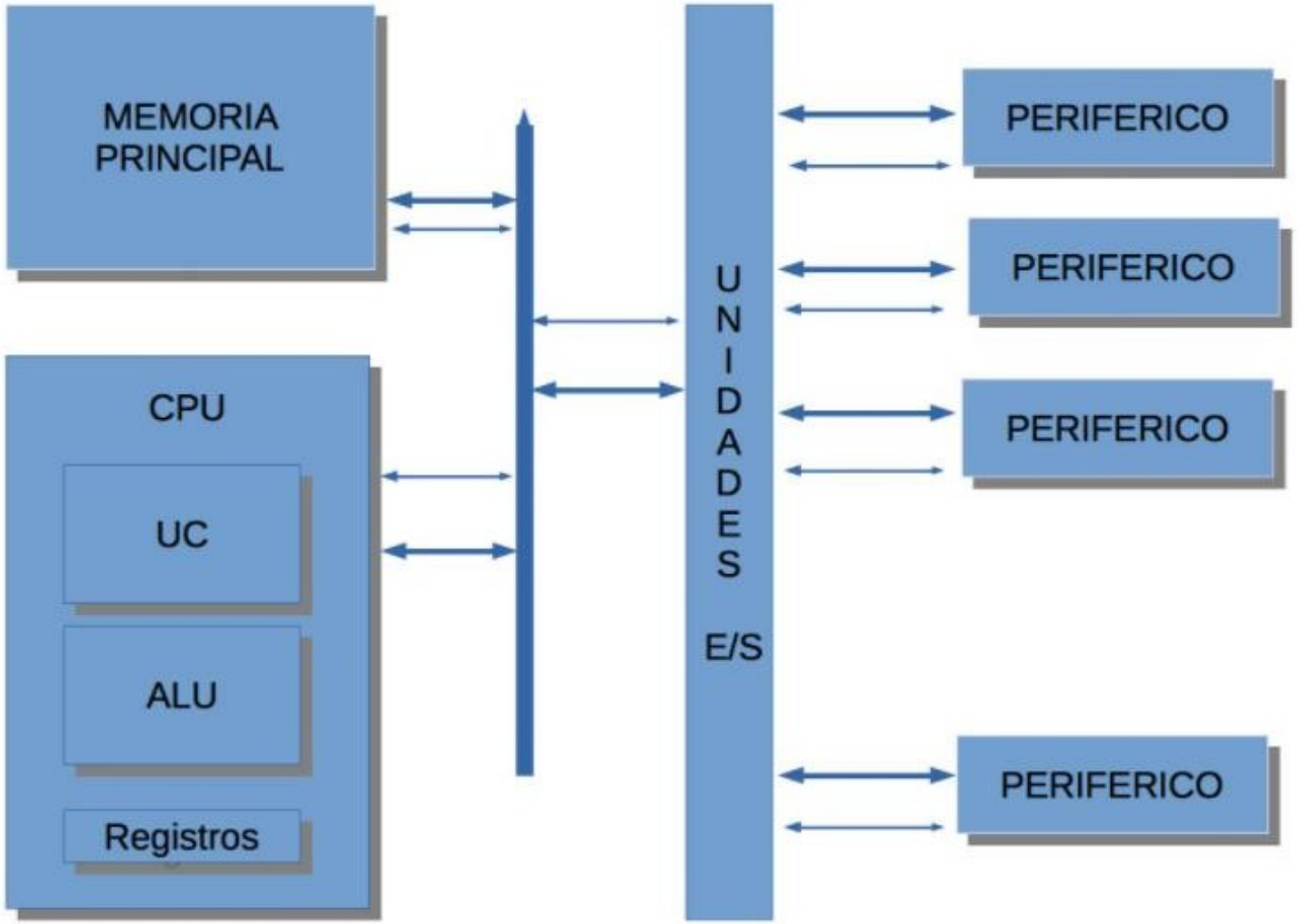
- Elementos de un proceso.
- Estados de un proceso. Cambios de estado.
- Planificación de procesos por el sistema operativo.

- Hilos:
  - Concepto y características.
  - Hilos vs. procesos.
- Sistemas multitarea
- Gestión de procesos.
- Programación de aplicaciones multiproceso

- Mapa conceptual



- Arquitectura de **Von Neumann**



▪ Programas y procesos

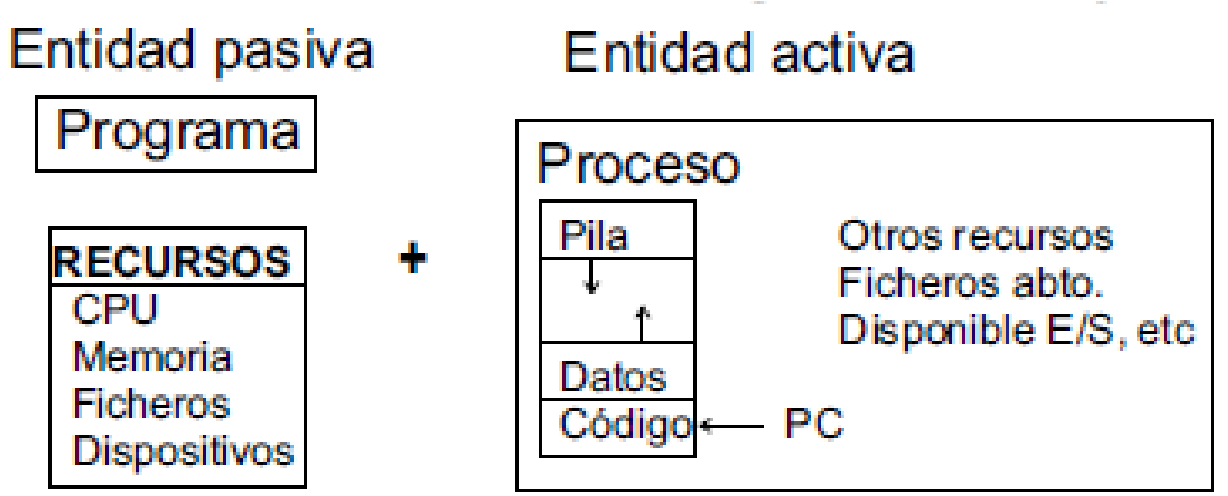
➢ Programa: conjunto de instrucciones que se pueden ejecutar en una máquina.

➢ Proceso: instancia de un programa en ejecución.

La ejecución de un programa empieza con la creación y ejecución de un proceso. Necesita ciertos recursos que se asignan cunado se crea o durante su ejecución:

- Tiempo de ejecución
- Memoria
- Acceso a archivos

y a dispositivos de E/S



▪ Programas y procesos

Para entender mejor el concepto de proceso:

– Los procesos tienen un carácter secuencial:
– Un proceso en su ejecución puede generar más de un proceso (llamada fork).
– Dos procesos pueden asociarse al mismo programa.

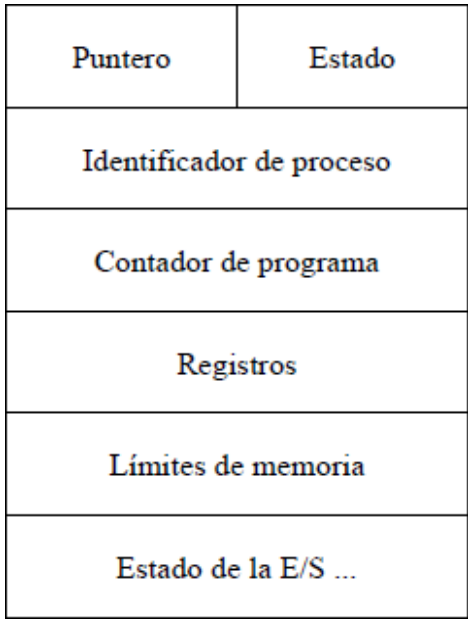
- Proceso: Unidad de trabajo del sistema.
  – En general, habrá más de un proceso ejecutándose concurrentemente.

- Procesos de usuario y procesos del sistema.

- El sistema operativo se encargará de:
  – La creación y eliminación de procesos.
  – La planificación de procesos.
  – La sincronización, comunicación y manejo de bloqueos mutuos entre procesos.

• Bloque de control de procesos (Process Control Block - PCB)

Toda la información relativa a un proceso se guarda en el PCB, si el mismo programa se ejecuta varias veces se creará cada vez un nuevo proceso con un nuevo PCB.



Estado actual del proceso.

Contador de programa: indica la dirección de la siguiente instrucción que se ejecutará de ese proceso.

Registros de CPU: acumuladores, registros índice, punteros de pila y registros generales.

Información de planificación de CPU: prioridad del proceso, punteros a colas de planificación, etc.

Información de gestión de memoria: valor de los registros de base y límite, tabla de páginas o tabla de segmentos.

Información de contabilidad: tiempo de CPU, tiempo consumido, números de procesos, etc.
Información de estado de E/S: dispositivos de E/S asignados a este proceso, lista de archivos abiertos, etc

Procesos en Linux

Linux https://www.webminal.org/terminal/
Process Explorer

*Ps -fu **profesor***

Campo	Significado
UID	ID del usuario propietario del proceso
PID	ID del proceso
PPID	ID del padre del proceso
STIME	Hora de comienzo
TTY	Terminal de control
C	Utilización del preprocesador de C para la administración de procesos
TIME	Tiempo acumulado de CPU
CMD	Nombre del comando

Procesos en Linux

Pstree -p **ps**

```
$pstree -p
?(1)─┬─?(32764)──sh(32765)
     ├─?(32752)──sh(32758)
     ├─?(32239)──sh(32352)
     ├─?(32241)──sh(32256)
     ├─?(29821)──sh(29898)
     ├─?(28833)──sh(28834)
     ├─?(28677)──bash(28678)
     ├─?(28556)──sh(28606)─┬─nano(30119)
     │                     └─sh(29734)──mc(29738)
     ├─?(25921)──sh(25999)
     ├─?(24833)──sh(24834)
     ├─?(23483)──sh(23484)
     ├─?(22250)──sh(22251)
     ├─?(21694)──sh(21711)
     ├─?(19784)──sh(19785)
     ├─?(19147)──sh(19148)──xxd(19193)
     ├─?(18744)──sh(18878)
     ├─?(17730)──sh(17734)
     ├─?(16923)──sh(16928)
     ├─?(11497)──sh(11520)
     ├─?(10967)──sh(10968)
     ├─?(9112)──sh(9192)
     ├─?(9108)──sh(9115)
     ├─?(9023)──sh(9029)
     ├─?(8958)──bash(8959)──bash(10593)
     ├─?(8806)──sh(8896)
     ├─?(8659)──sh(8660)
     ├─?(8394)──sh(8398)──python(8463)
```

```
~]$ps
  PID TTY          TIME CMD
 5490 pts/78   00:00:00 sh
11326 pts/78   00:00:00 sleep
11333 pts/78   00:00:00 ps
$kill 11326
```

Servicios en Linux

```
[root@tecmint ~]#  systemctl  list-units --type=service
  UNIT                              LOAD   ACTIVE SUB     DESCRIPTION
  auditd.service                    loaded active running Security Auditing Service
  chronyd.service                   loaded active running NTP client/server
  crond.service                     loaded active running Command Scheduler
  dbus.service                      loaded active running D-Bus System Message Bus
  dracut-shutdown.service           loaded active exited  Restore /run/initramfs on shutdown
  firewalld.service                 loaded active running firewalld - dynamic firewall daemon
  getty@tty1.service                loaded active running Getty on tty1
  gssproxy.service                  loaded active running GSSAPI Proxy Daemon
```

VIEWING systemd INFORMATION

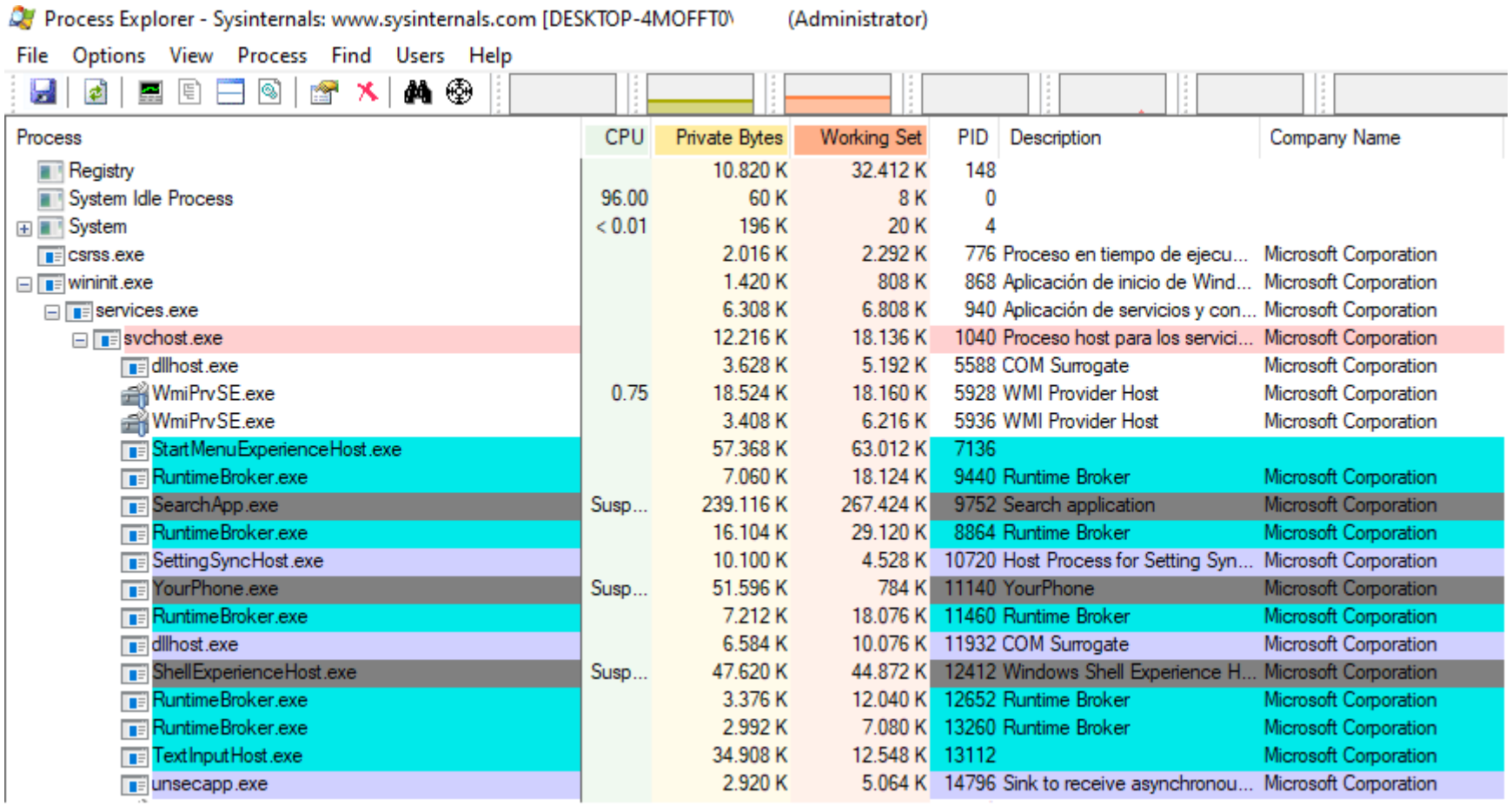
systemctl list-dependencies	Show a unit's dependencies
systemctl list-sockets	List sockets and what activates
systemctl list-jobs	View active systemd jobs
systemctl list-unit-files	See unit files and their states
systemctl list-units	Show if units are loaded/active
systemctl get-default	List default target (like run level)

WORKING WITH SERVICES

systemctl stop *service*	Stop a running service
systemctl start *service*	Start a service
systemctl restart *service*	Restart a running service
systemctl reload *service*	Reload all config files in service
systemctl daemon-reload	Must run to reload changed unit files
systemctl status *service*	See if service is running/enabled
systemctl --failed	Shows services that failed to run
systemctl reset-failed	Resets any units from failed state
systemctl enable *service*	Enable a service to start on boot
systemctl disable *service*	Disable service--won't start at boot
systemctl show *service*	Show properties of a service (or other unit)
systemctl edit *service*	Create snippit to drop in unit file
systemctl edit --full *service*	Edit entire unit file for service
systemctl -H *host* **status network**	Run any systemctl command remotely

https://access.redhat.com/sites/default/files/attachments/12052018_systemd_6.pdf

Procesos en Windows



Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
Registry		10.820 K	32.412 K	148		
System Idle Process	96.00	60 K	8 K	0		
System	< 0.01	196 K	20 K	4		
csrss.exe		2.016 K	2.292 K	776	Proceso en tiempo de ejecu...	Microsoft Corporation
wininit.exe		1.420 K	808 K	868	Aplicación de inicio de Wind...	Microsoft Corporation
services.exe		6.308 K	6.808 K	940	Aplicación de servicios y con...	Microsoft Corporation
svchost.exe		12.216 K	18.136 K	1040	Proceso host para los servici...	Microsoft Corporation
dllhost.exe		3.628 K	5.192 K	5588	COM Surrogate	Microsoft Corporation
WmiPrvSE.exe	0.75	18.524 K	18.160 K	5928	WMI Provider Host	Microsoft Corporation
WmiPrvSE.exe		3.408 K	6.216 K	5936	WMI Provider Host	Microsoft Corporation
StartMenuExperienceHost.exe		57.368 K	63.012 K	7136		
RuntimeBroker.exe		7.060 K	18.124 K	9440	Runtime Broker	Microsoft Corporation
SearchApp.exe	Susp...	239.116 K	267.424 K	9752	Search application	Microsoft Corporation
RuntimeBroker.exe		16.104 K	29.120 K	8864	Runtime Broker	Microsoft Corporation
SettingSyncHost.exe		10.100 K	4.528 K	10720	Host Process for Setting Syn...	Microsoft Corporation
YourPhone.exe	Susp...	51.596 K	784 K	11140	YourPhone	Microsoft Corporation
RuntimeBroker.exe		7.212 K	18.076 K	11460	Runtime Broker	Microsoft Corporation
dllhost.exe		6.584 K	10.076 K	11932	COM Surrogate	Microsoft Corporation
ShellExperienceHost.exe	Susp...	47.620 K	44.872 K	12412	Windows Shell Experience H...	Microsoft Corporation
RuntimeBroker.exe		3.376 K	12.040 K	12652	Runtime Broker	Microsoft Corporation
RuntimeBroker.exe		2.992 K	7.080 K	13260	Runtime Broker	Microsoft Corporation
TextInputHost.exe		34.908 K	12.548 K	13112		Microsoft Corporation
unsecapp.exe		2.920 K	5.064 K	14796	Sink to receive asynchronou...	Microsoft Corporation

Procesos en Windows

```
tasklist

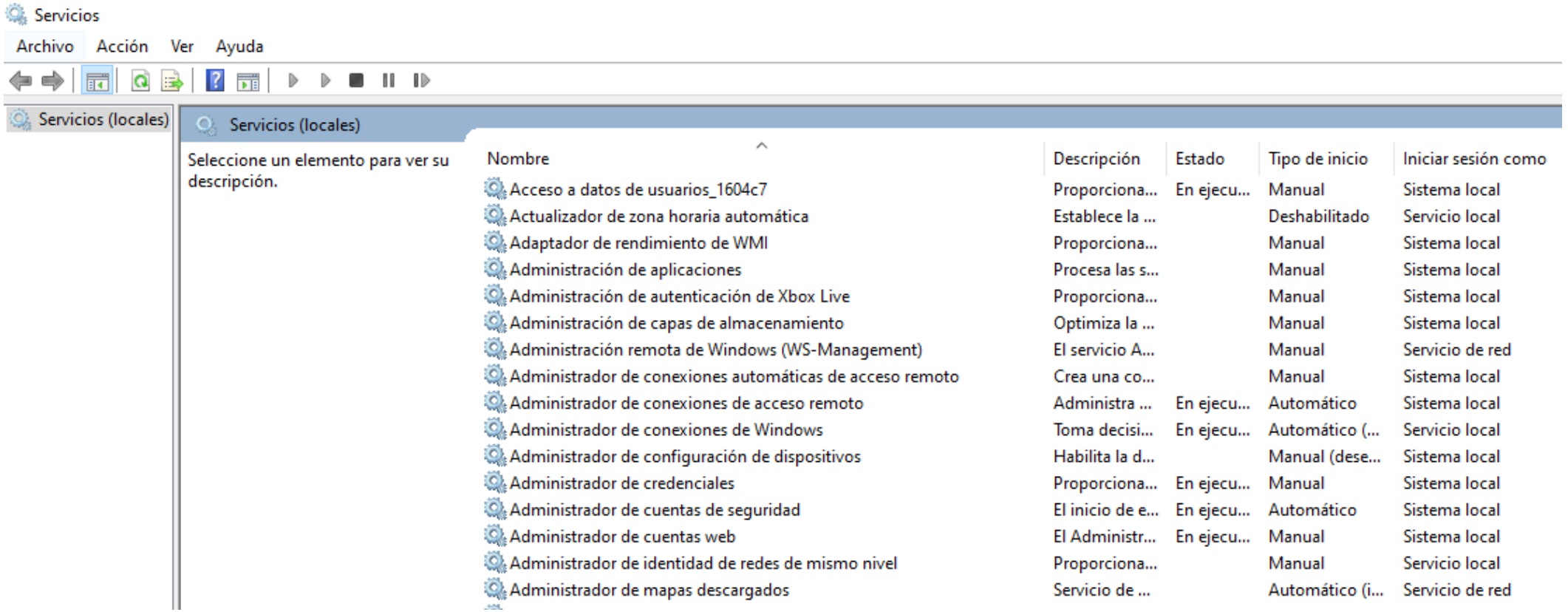
Nombre de imagen              PID Nombre de sesión Núm. de ses Uso de memor
========================= ======== ================ =========== ============
System Idle Process             0 Services                   0         8 KB
System                          4 Services                   0        20 KB
Registry                      148 Services                   0    32.420 KB
smss.exe                      536 Services                   0       520 KB
csrss.exe                     776 Services                   0     2.296 KB
wininit.exe                   868 Services                   0       808 KB
csrss.exe                     876 Console                    1     3.044 KB
services.exe                  940 Services                   0     6.848 KB
lsass.exe                     960 Services                   0    13.508 KB
winlogon.exe                  480 Console                    1     5.976 KB
svchost.exe                  1040 Services
fontdrvhost.exe              1076 Console
fontdrvhost.exe              1080 Services
WUDFHost.exe                 1144 Services
svchost.exe                  1212 Services
svchost.exe                  1268 Services
dwm.exe                      1348 Console
```

```
tasklist /svc /fi "imagename eq svchost.exe"

Nombre de imagen              PID Servicios
========================= ======== ============================================
svchost.exe                  1040 BrokerInfrastructure, DcomLaunch, PlugPlay,
                                  Power, SystemEventsBroker
svchost.exe                  1212 RpcEptMapper, RpcSs
svchost.exe                  1268 LSM
svchost.exe                  1556 EventLog
svchost.exe                  1580 Schedule
svchost.exe                  1616 NcbService
svchost.exe                  1624 ProfSvc
svchost.exe                  1632 TimeBrokerSvc
svchost.exe                  1728 nsi
svchost.exe                  1736 CoreMessagingRegistrar
svchost.exe                  1824 UserManager
svchost.exe                  1844 hidserv
svchost.exe                  1872 Dhcp
svchost.exe                  1836 NlaSvc
svchost.exe                  2136 netprofm
svchost.exe                  2196 DispBrokerDesktopSvc
```

Servicios en Windows

SERVICES.MSC

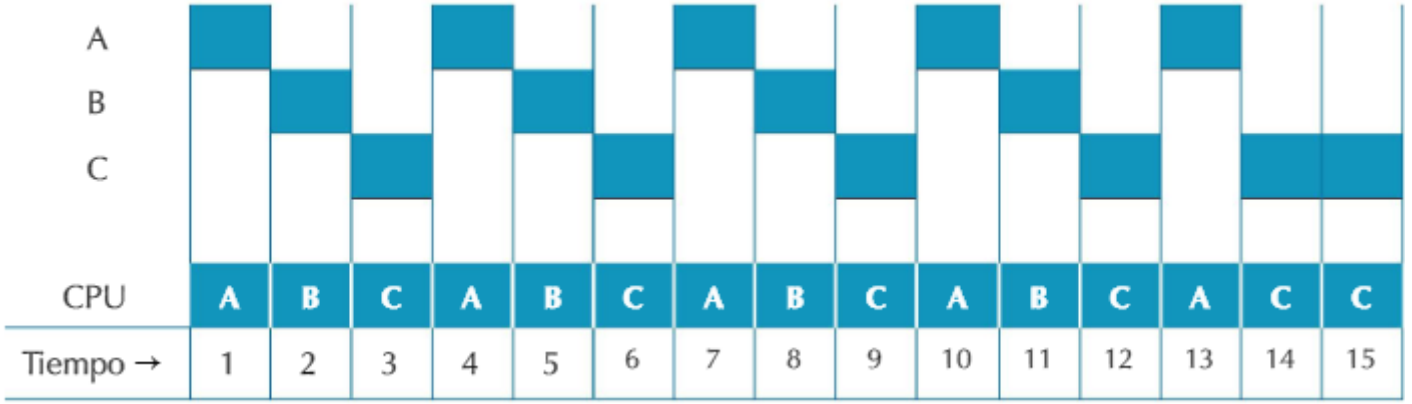


- **Multitarea**

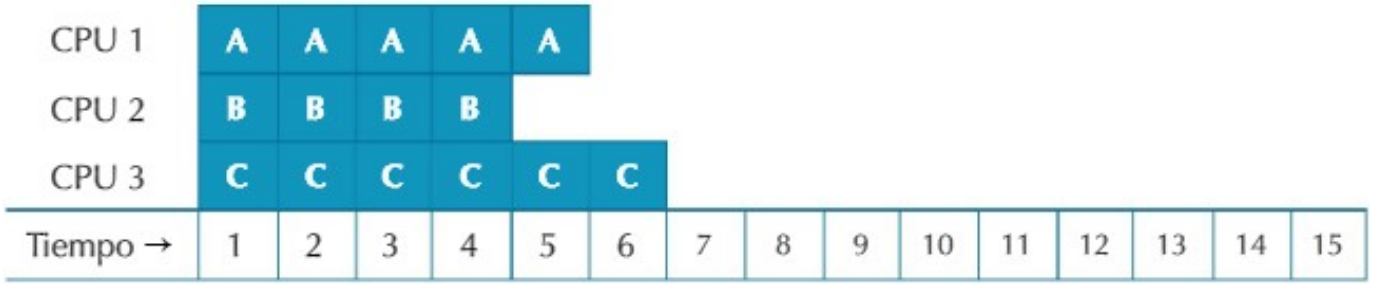
Ejecución simultanea **de más de un proceso** en un procesador en un intervalo de tiempo.

CPU	A	A	A	A	A	B	B	B	B	C	C	C	C	C	C
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Sin multitarea



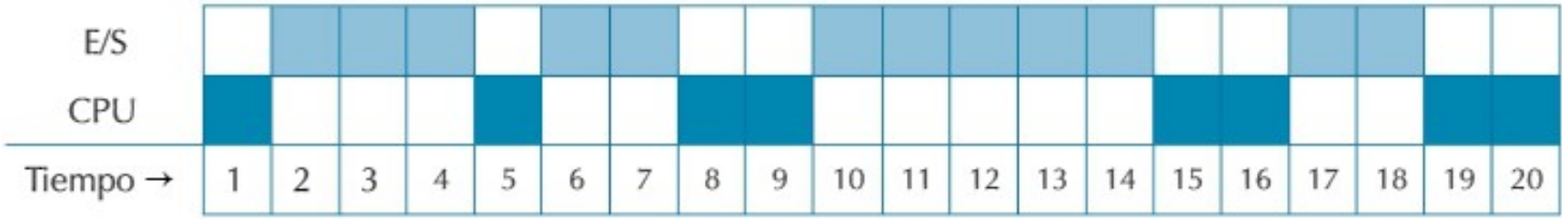
Con multitarea
(cambio de contexto)



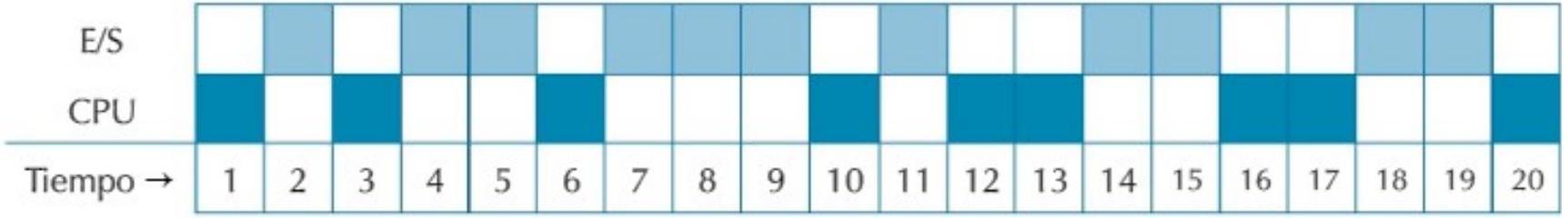
Con multitarea y multiprocesador

- **Multitarea**

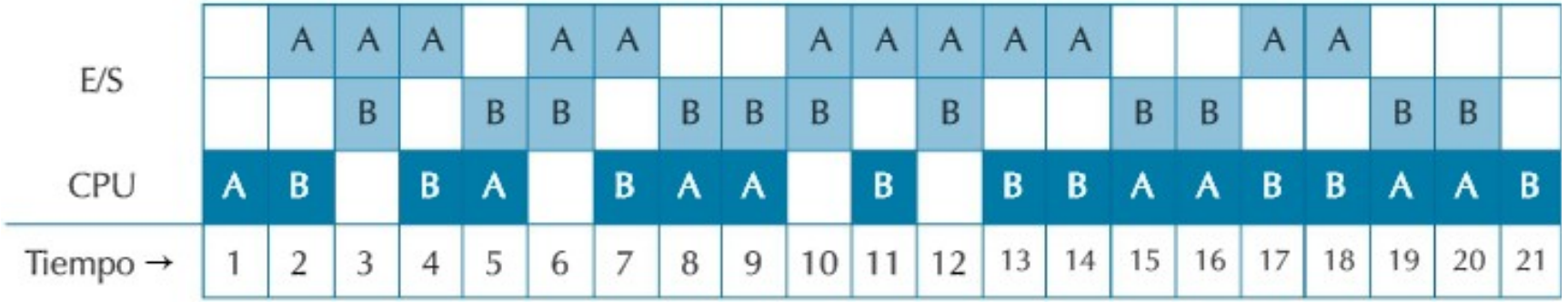
Proceso A



Proceso B

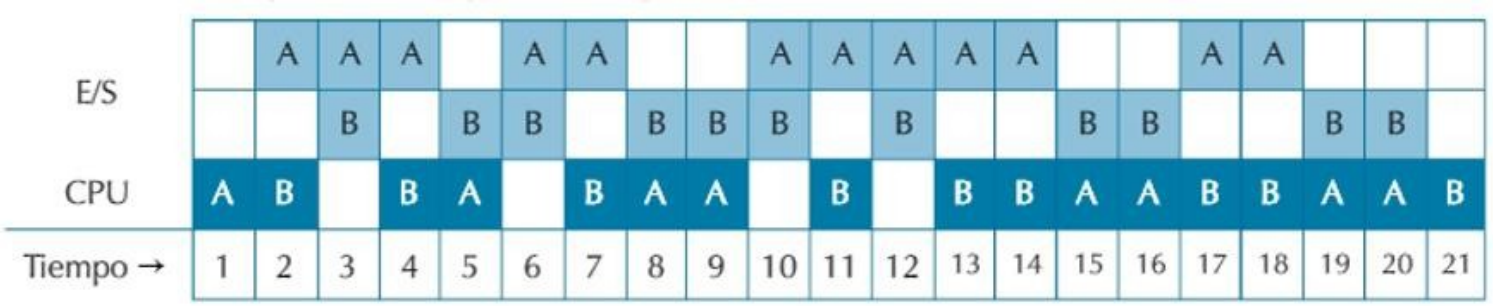


Multitarea con procesos A y B en un procesador

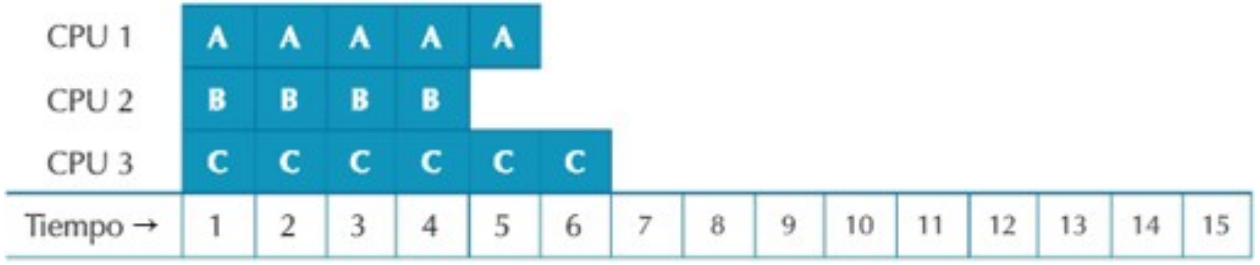


Sistema monoprocesador y multiprocesador

En un sistema **monoprocesador** (un solo procesador) la forma de gestionar los procesos para ejecutarlos de forma concurrente recibe el nombre **multiprogramación**

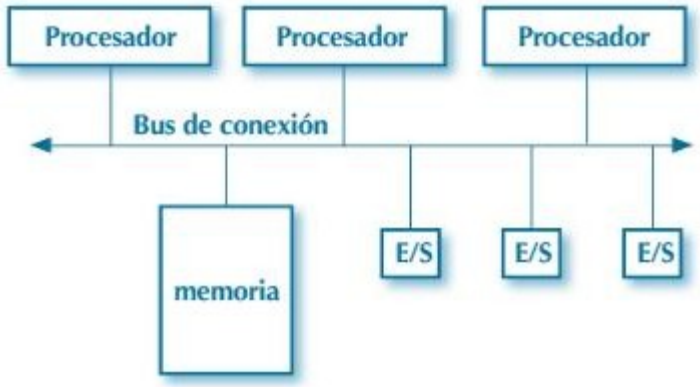


En un sistema **multiprocesador** (existe más de un procesador) se pueden tener un proceso en cada procesador esto permite paralelismo real, a la gestión de varios procesos dentro de un sistema multiprocesador se le denomina **multiproceso**.

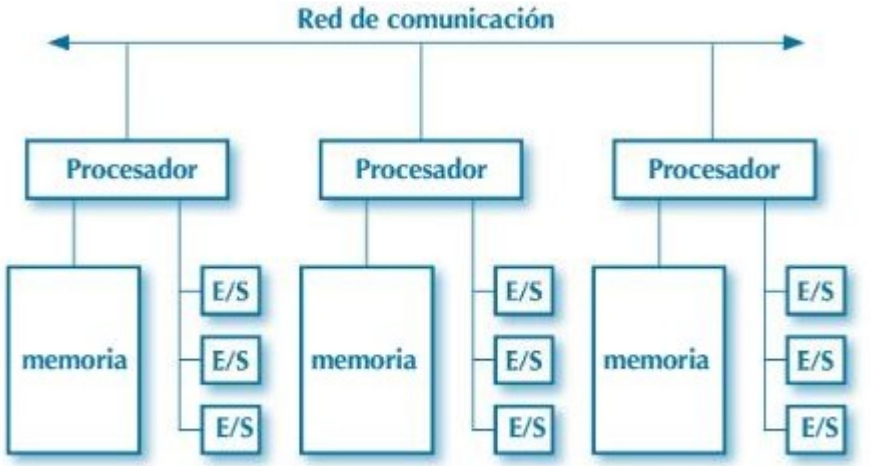


Sistema multiprocesador

Fuertemente acoplados(simétricos y asimétricos):



Débilmente acoplados:



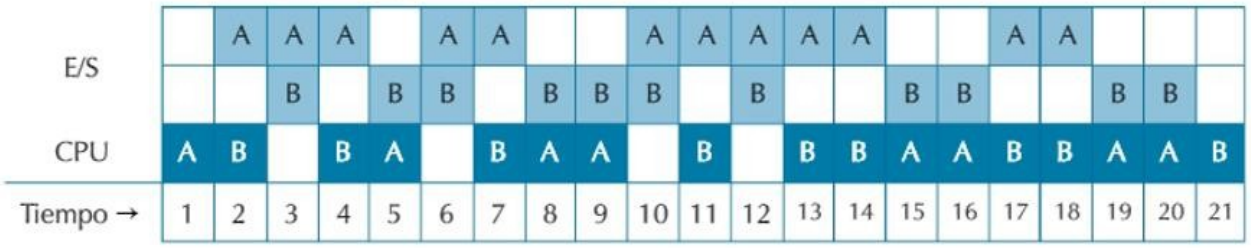
Sistema distribuido

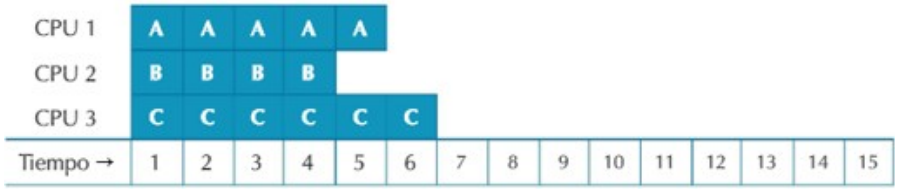


- **Programación concurrente**

Programación concurrente: cuando se ejecutan varios procesos concurrente en un sistema. También se refiere a las técnicas que permiten desarrollar programas que utilizan procesos concurrentes.

Programación paralela: varios procesos concurrentes en un sistemas multiprocesador. Se pueden ejecutar con un paralelismo real, de forma simultanea.

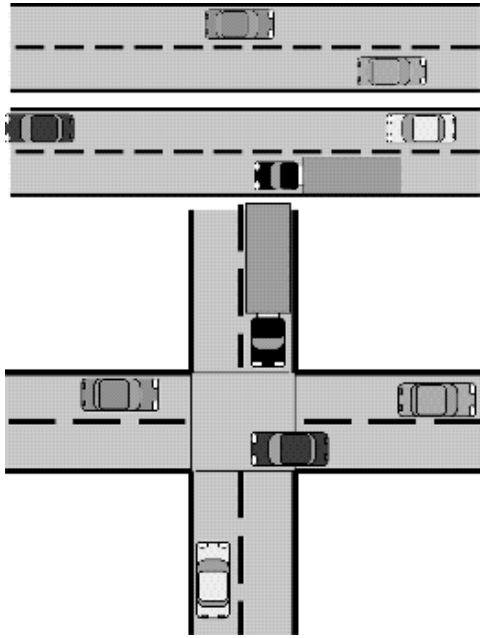




- **Programación concurrente**

Ventajas:

- Mejor aprovechamiento de la CPU
- Velocidad de ejecución



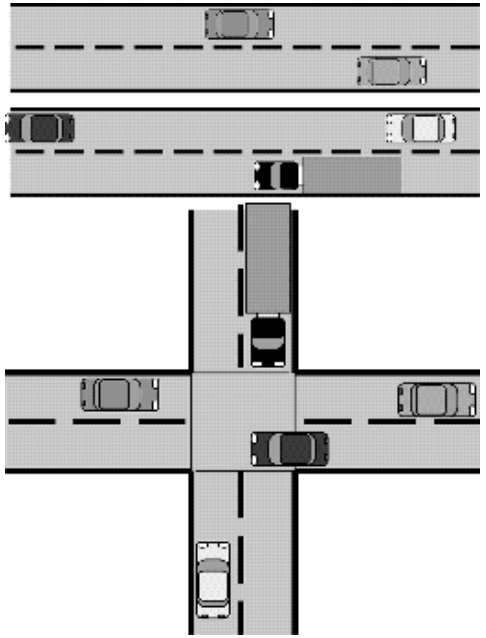
Inconvenientes

- Dificultad de implementar mecanismos de sincronización y comunicación entre procesos.

- **Programación concurrente**

Ventajas:

- Mejor aprovechamiento de la CPU
- Velocidad de ejecución

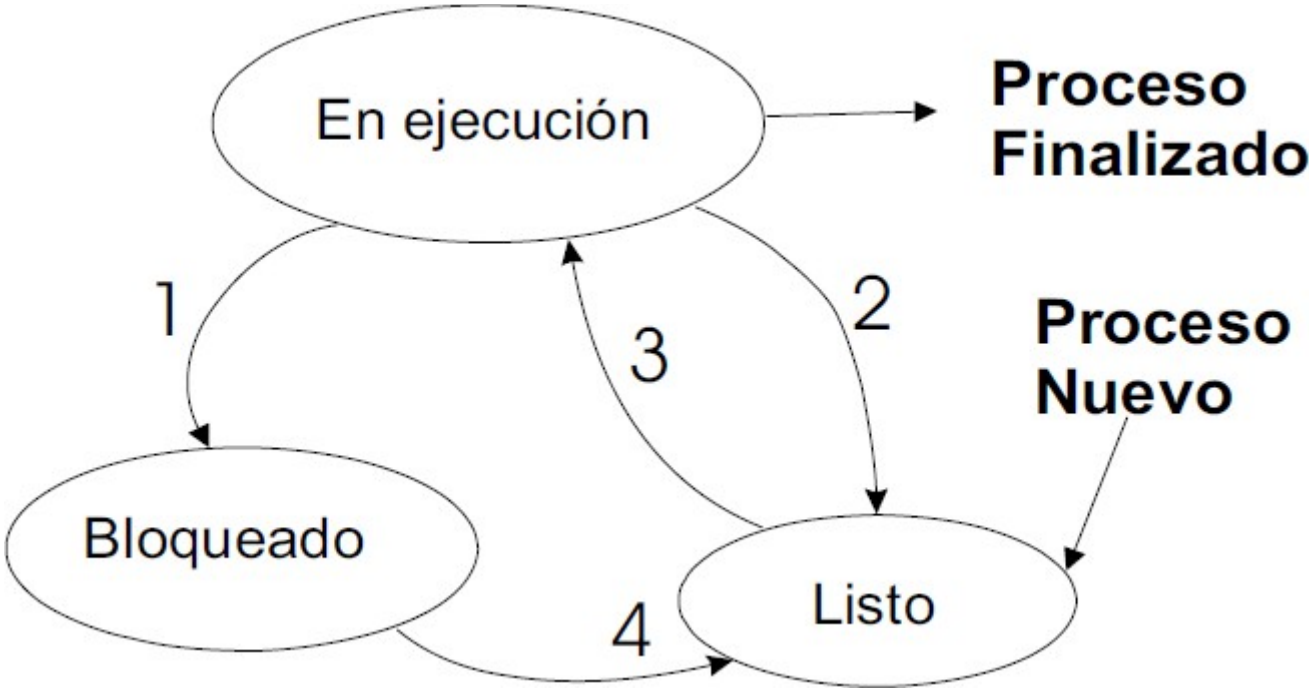


Inconvenientes

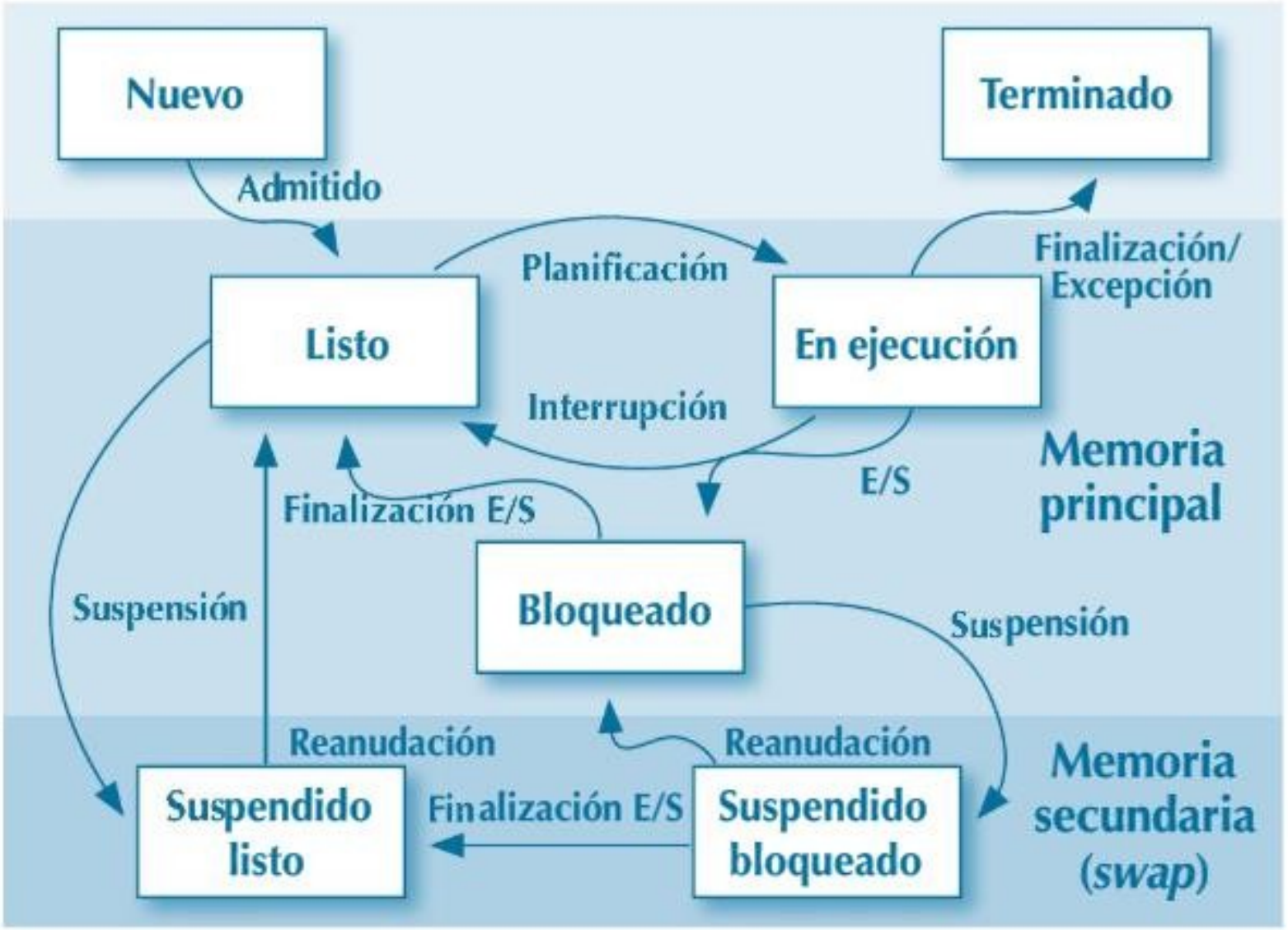
- Dificultad de implementar mecanismos de sincronización y comunicación entre procesos.

- **Estados de un proceso**

1. Pasa a esperar un suceso (E/S) y se bloquea.
2. Expulsión de un proceso de la CPU.
3. El planificación elige otro proceso.
4. El suceso (E/s) que esperaba el proceso acaba.



- **Estados de un proceso**

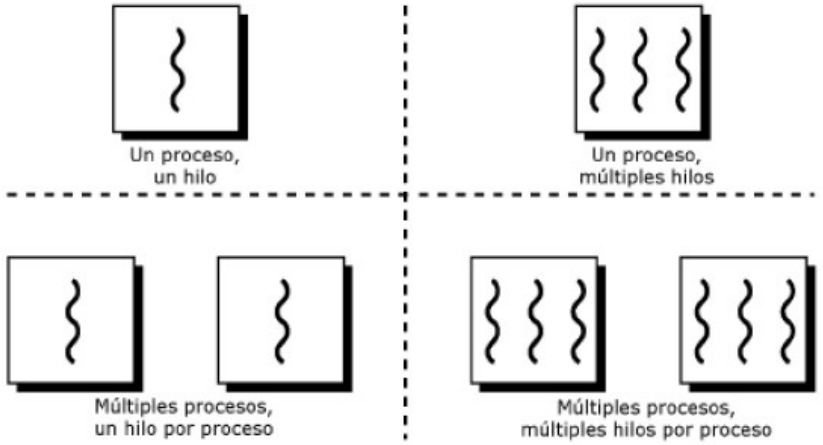


- **Hilos y procesos**

Un hilo(proceso ligero) es una **unidad básica de utilización de la CPU**, y consiste en un contador de programa, un juego de registros y un espacio de pila

Los hilos dentro de una misma aplicación comparten
- La sección de código.
- La sección de datos.
- Los recursos del SO(archivos abiertos y señales).

Un proceso tradicional o pesado es igual a una tarea con un solo hilo.



▪ Hilos y procesos

Los hilos permiten la ejecución concurrentede varias secuencias de instrucciones asociadas a diferentes funciones dentro de un mismo proceso, compartiendoun mismo espacio de direccionesy las mismas estructuras de datosdel núcleo.

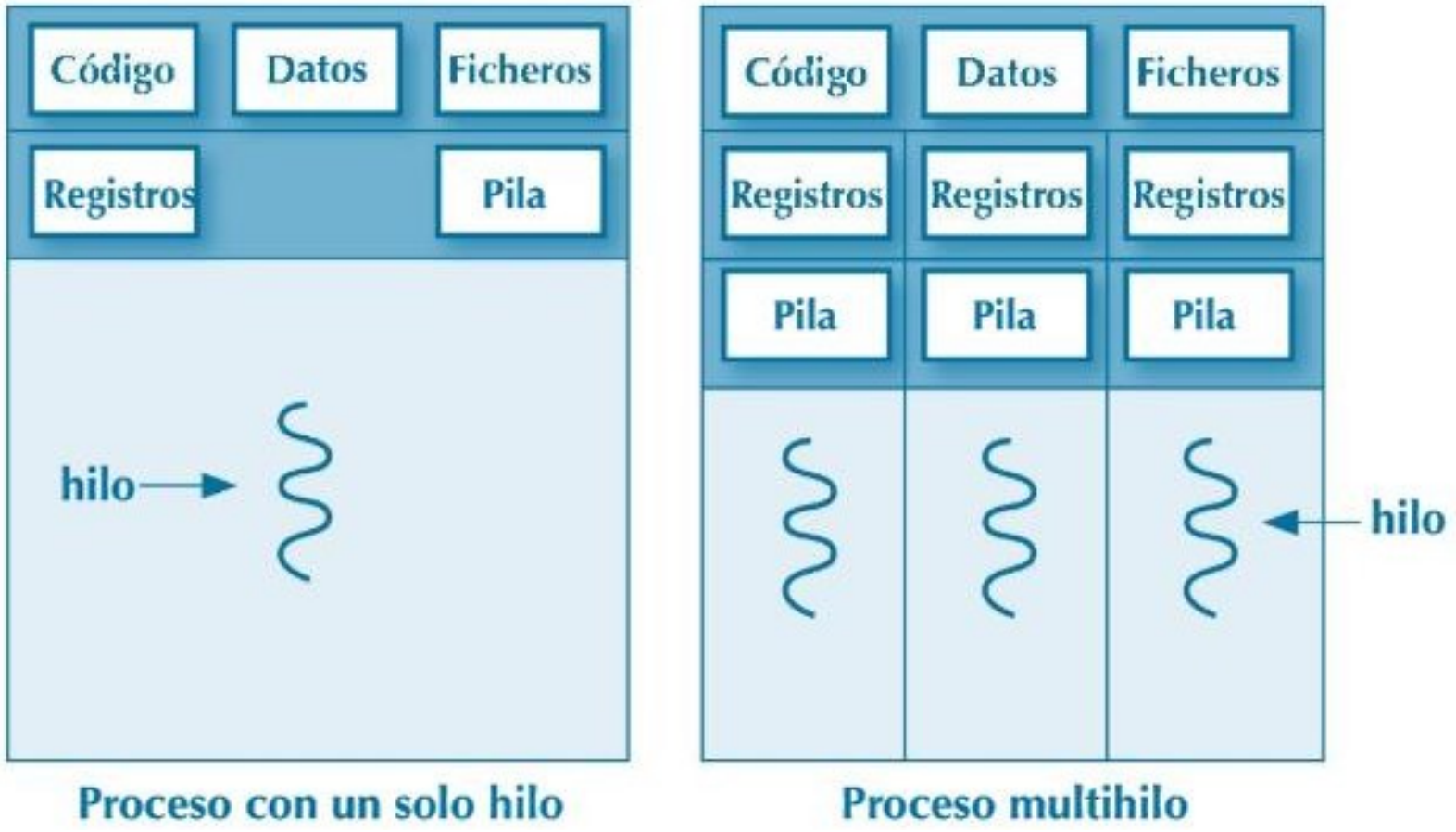
Recursos compartidos entre los hilos:

- Código(instrucciones).
- Variables globales.
- Ficheros y dispositivos abiertos.

Recursos no compartidos entre los hilos:

- **Contador del programa** (cada hilo puede ejecutar una sección distinta de código).
- **Registros de CPU**.
- Pila **para las variables locales** de los procedimientos a las que se invoca después de crear un hilo.
- **Estado**: distintos hilos pueden estar en ejecución, listos o bloqueados esperando un evento.

- **Hilos y procesos**



- **Clases Process, Runtime, ProcessBuilder**

- **Process**

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Process.html

Método	Funcionalidad
`void destroy()` `public Process destroyForcibly()`	Termina el proceso. El primer método permite una terminación limpia y ordenada del proceso. El segundo lo termina inmediatamente.
`int exitValue()`	Devuelve el valor de salida, o código de retorno, del proceso. Por convención, un valor 0 indica terminación normal, y otro valor se interpretará como un código de error. Se puede terminar un programa en Java con un código de retorno distinto de 0 con `System.exit(código)`.
`ProcessHandle.Info info()`	Devuelve la información actual del proceso.
`boolean isAlive()`	Comprueba si el proceso está vivo.
`long pid()`	Devuelve el PID o identificador de proceso.
`int waitFor()`	Hace que el hilo en ejecución espere hasta que el proceso haya terminado. Devuelve el valor de salida del proceso. Un valor cero se entiende que corresponde a una ejecución sin errores, mientras que un valor distinto de cero corresponde a un código de error. Si el proceso es de un programa en Java, es el valor devuelto por `System.exit()`, o cero si no se terminó la ejecución con `System.exit()`.
`boolean waitFor(long timeout, TimeUnit unit)`	Hace que el hilo en ejecución espere hasta que el proceso haya terminado, durante un tiempo máximo indicado por `timeout`. Devuelve `true` si el proceso ha terminado por sí mismo antes del tiempo máximo indicado, y `false` en caso contrario.

- **Clases Process, Runtime, ProcessBuilder**

Clase Process (abstracta) tiene métodos para lanzar procesos, obtener información y controlar su ejecución

Subclases:

- **Runtime**

Método	Funcionalidad
`static Runtime getRuntime()`	Devuelve el objeto `Runtime` asociado con la aplicación de Java actual.

`Process exec(String command)`
`Process exec(String[] comandoYArgs)`
`Process exec(String[] comandoYArgs, String[] envp)`
`Process exec(String[] cmdarray, String[] envp, File dir)`
	Ejecuta un comando. Algunas variantes permiten especificar el entorno de ejecución y el directorio para ejecutar el comando.

`void exit(int status)` `void halt(int status)`	Termina la ejecución de la máquina virtual. El primer método lo hace de manera ordenada, y el segundo, de manera abrupta.
`int availableProcessors()`	Devuelve el número de procesadores disponibles para la máquina virtual de Java.
`long freeMemory()`	Devuelve la cantidad de memoria disponible para la máquina virtual.

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Runtime.html

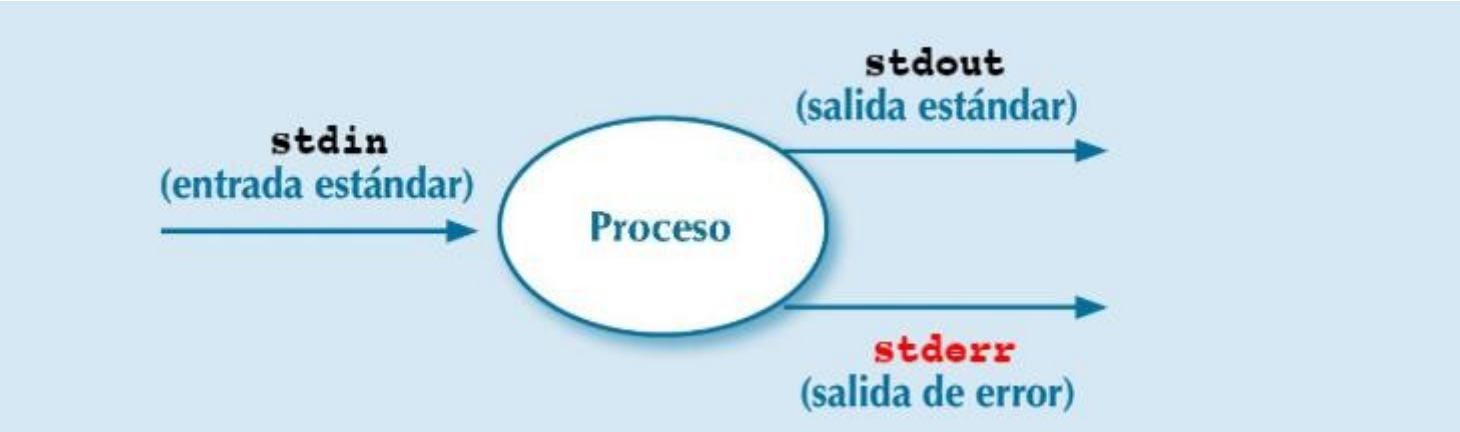
- **Clases Process, Runtime, ProcessBuilder**

- **ProcessBuilder**

Método	Funcionalidad
`ProcessBuilder(String... comando)` `ProcessBuilder(List<String> comando)`	Construye un `ProcessBuilder` para un comando con los correspondientes argumentos de línea de comandos, en su caso.
`ProcessBuilder directory(File directorio)` `File directory()`	El primer método establece el directorio de trabajo del proceso, y el segundo método lo devuelve. Por defecto, el segundo método normalmente devuelve `null`. Si es así, se puede obtener el directorio de ejecución del proceso actual con `System.getProperty("user.dir")`.
`Map<String,String> environment()`	Devuelve el entorno de trabajo del proceso, que es una lista de asignación de valores para variables de entorno.
`Process start()`	Crea e inicia un proceso.

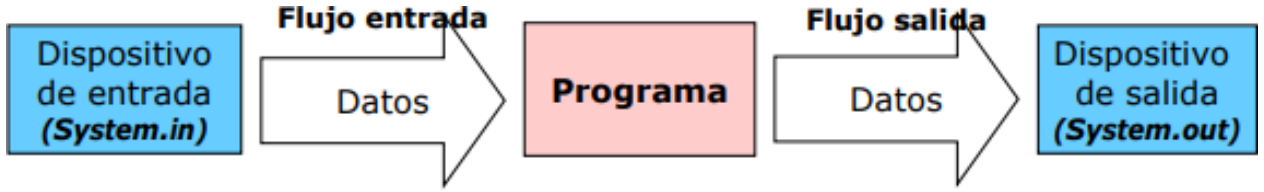
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ProcessBuilder.html

Entrada/salida E/S



En Java se accede a la E/S estándar a través de campos estáticos de la clase java.lang.System

- **System.in** implementa la entrada estándar
- **System.out** implementa la salida estándar
- **System.err** implementa la salida de error



Entrada/salida E/S

- **System.in**
  - Instancia de la clase InputStream: flujo de bytes de entrada
  - Métodos:
    - read() permite leer un byte de la entrada como entero
    - skip(n) ignora n bytes de la entrada
    - available() número de bytes disponibles para leer en la entrada

- **System.out**
  - Instancia de la clase PrintStream: flujo de bytes de salida
  - Métodos:
    - print(), println() • flush() vacía el buffer de salida escribiendo su contenido
- **System.err**
  - Funcionamiento similar a System.out
  - Se utiliza para enviar mensajes de error (por ejemplo a un fichero de log o a la consola)

- **Redirección E/S**

	Fichero	Ejemplo
A entrada estándar desde	*proceso* < *fichero*	`grep localhost < /etc/hosts`
De salida estándar hacia	*proceso* > *fichero* *proceso* >> *fichero*	`ls -l > listafich.txt` `echo –fin- >> listafich.txt`
De salida de error hacia	*proceso* 2> *fichero* *proceso* 2>> *fichero*	`ls /et 2> errores.log` `ls /etcaetera 2>> errores.log`

(*Nota:* Para desechar salida, se puede redirigir a `/dev/null`)

		Ejemplo
De salida estándar de un proceso a entrada estándar de otro	*proceso1* | *proceso2*	`head -4 /etc/fstab | tail -1`

- **Redirección E/S**

	Fichero	Ejemplo
A entrada estándar desde	*proceso* < *fichero*	`grep localhost < /etc/hosts`
De salida estándar hacia	*proceso* > *fichero* *proceso* >> *fichero*	`ls -l > listafich.txt` `echo –fin- >> listafich.txt`
De salida de error hacia	*proceso* 2> *fichero* *proceso* 2>> *fichero*	`ls /et 2> errores.log` `ls /etcaetera 2>> errores.log`

(*Nota:* Para desechar salida, se puede redirigir a `/dev/null`)

		Ejemplo
De salida estándar de un proceso a entrada estándar de otro	*proceso1* | *proceso2*	`head -4 /etc/fstab | tail -1`

- **Redirección E/S**

ProcessBuilder

Métodos:

-InheritIO(): la entrada y salida estándar y de error de los procesos creados se enlazan con el proceso actual.

- **RedirectInput():** redirige la entrada estándar del proceso padre con redirect.INHERIT, el contenido de un fichero con un objeto File.

- **RedirectOUtput():** redirige la salida estándar del proceso padre con **redirect.INHERIT**, a un fichero clase File borrando el contenido, **Redircet.appendTo(f)** añadiendo al final, **Redirect.DISCARD** descartándolo

- **Redirección E/S**

ProcessBuilder

Métodos:

-InheritIO(): la entrada y salida estándar y de error de los procesos creados se enlazan con el proceso actual.

- **RedirectInput():** redirige la entrada estándar del proceso padre con redirect.INHERIT, el contenido de un fichero con un objeto File.

- **RedirectOUtput():** redirige la salida estándar del proceso padre con **redirect.INHERIT**, a un fichero clase File borrando el contenido, **Redircet.appendTo(f)** añadiendo al final, **Redirect.DISCARD** descartándolo

Runtime

```
import java.io.IOException;
import java.util.concurrent.TimeUnit;

public class LanzaProceso {

public static void main(String[] args) {

Runtime r =
Runtime.getRuntime(); String
comando = "notepad"; Process p;

      try {
            p = r.exec(comando);

            p.waitFor(3, TimeUnit.SECONDS);

            p.destroy();

      } catch (IOException e1) {

      System.out.println("Error en: " + comando);
      e1.printStackTrace();

      } catch (InterruptedException e1) {

      System.err.println("Proceso interrumpido");
      e1.printStackTrace();

      }catch (Exception e) {
      e.printStackTrace();
      }
}
}
```

ProcessBuilder

```
import java.io.IOException;

public class LanzaProceso {

  public static void main(String[] args) {

String proceso = "notepad";

    ProcessBuilder pb = new ProcessBuilder(proceso);
    pb.inheritIO();

    try {
      Process p = pb.start();
      int codRet = p.waitFor();

      System.out.println("La ejecución de " + proceso
              + " devuelve " + codRet
              + " " + (codRet == 0 ? "(ejecución correcta)" : "(ERROR)")
      );
    } catch (IOException e) {
      System.err.println("Error durante ejecución del proceso");
      System.exit(2);
    } catch (InterruptedException e) {
      System.err.println("Proceso interrumpido");
      System.exit(3);
    }

  }

}
```

Material de estudio

- https://docs.oracle.com/javase/tutorial/essential/concurrency/
- Java ProcessBuilder tutorial
- Java ProcessBuilder
- Programación concurrente
- Java concurrent tutorial
- https://www.udemy.com/course/gnulinux-desde-cero/